PROGETTO FINALE DI PROGRAMMAZIONE

Sistema di Gestione delle Spese Personali e del Budget

Studente: Salvatore C.

Linguaggio: C++ | Database: SQLite

Repository GitHub del progetto:

https://github.com/csalvatore2/PersonalExpenseSystem

1. Schema Logico del Database

Il database è composto da tre tabelle: Categorie, Transazioni e Budget. Le ultime due fanno riferimento a Categorie tramite chiave esterna.

Transazioni			
🔑	id	INT	PRIMARY KEY, AUTOINCREMENT
○	Importo_max	DECIMAL (10,2)	NOT NULL, CHECK (importo_max > 0)
○	Mese	INT	NOT NULL, CHECK (Mese>=1 AND Mese <= 12)
○	Anno	INT	NOT NULL, CHECK (Anno > 0)
🔗	Categoria	INT	NOT NULL, FOREIGN KEY -> Categorie(id)
UNIQUE (Mese, Anno, Categoria)			

Categorie			
🔑	id	INT	PRIMARY KEY, AUTOINCREMENT
○	Nome	VARCHAR(255)	NOT NULL, UNIQUE

Budget			
🔑	id	INT	PRIMARY KEY, AUTOINCREMENT
○	Data	DATE	NOT NULL
○	Importo	DECIMAL (10,2)	NOT NULL, CHECK (importo > 0)
○	Descrizione	VARCHAR(800)	
🔗	Categoria	INT	NOT NULL, FOREIGN KEY -> Categorie(id)

2. Script SQL — Creazione del Database

Codice SQL completo per la creazione delle tabelle con tutti i vincoli di integrità.

```sql
-- GENERAZIONE_TABELLE

CREATE TABLE Categorie (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    Nome varchar(255) NOT NULL UNIQUE
);

CREATE TABLE Transazioni (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    Data date NOT NULL,
    Importo decimal(10, 2) NOT NULL CHECK (Importo > 0),
    Descrizione varchar(600),
    Categoria int NOT NULL,

    FOREIGN KEY (Categoria) REFERENCES Categorie(id)
);

CREATE TABLE Budget(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    Importo_max decimal(10, 2) NOT NULL CHECK (Importo_max > 0),
    Mese int NOT NULL CHECK (Mese >= 1 AND Mese <= 12),
    Anno int NOT NULL CHECK (Anno > 0),
    Categoria int NOT NULL,

    UNIQUE (Mese, Anno, Categoria),
    FOREIGN KEY (Categoria) REFERENCES Categorie(id)
);
```

Vincoli di integrità presenti

Vincolo	Tabella / Campo	Descrizione
PRIMARY KEY	Tutte le tabelle (id)	Identificatore univoco di ogni record
NOT NULL	Campi obbligatori	Impedisce valori nulli
UNIQUE	Categorie.Nome	Nessuna categoria duplicata
UNIQUE	Budget (Mese, Anno, Categoria)	Un solo budget per mese/ categoria
CHECK Importo>0	Transazioni.Importo	L'importo deve essere positivo
CHECK Importo>0	Budget.Importo_max	Il budget deve essere positivo
CHECK Mese 1..12	Budget.Mese	Valore mese valido
CHECK Anno>0	Budget.Anno	Anno positivo
FOREIGN KEY	Transazioni, Budget	Integrita referenziale verso Categorie(id)

3. Script SQL — Dati di Esempio (INSERT)

Dati di esempio per dimostrare il funzionamento del sistema. Il file completo è disponibile nel repository al percorso sql/popola_con_dati.sql.

```sql
INSERT INTO Categorie (Nome) VALUES ('Alimentari');
INSERT INTO Categorie (Nome) VALUES ('Abitazione');
INSERT INTO Categorie (Nome) VALUES ('Trasporti');
INSERT INTO Categorie (Nome) VALUES ('Svago');
INSERT INTO Categorie (Nome) VALUES ('Varie ed eventuali');
INSERT INTO Categorie (Nome) VALUES ('Abbonamenti');

INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (100.50, '2023-01-15', 1, 'Spesa settimanale');
INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (200.00, '2023-01-20', 2, 'Affitto');
INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (50.75,  '2023-01-25', 3, 'Benzina');
INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (25.40,  '2023-01-05', 1, 'Pane e latte');
INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (15.90,  '2023-01-07', 1, 'Frutta e verdura');
INSERT INTO Transazioni (Importo, Data, Categoria, Descrizione)
      VALUES (60.00,  '2023-01-10', 6, 'Abbonamento streaming mensile');

INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (300.00, 1, 2023, 1);
INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (800.00, 1, 2023, 2);
INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (150.00, 1, 2023, 3);
INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (100.00, 1, 2023, 4);
INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (280.00, 1, 2023, 5);
INSERT INTO Budget (Importo_max, Mese, Anno, Categoria)
      VALUES (12.00,  1, 2023, 6);
```

Tutte le query rimanenti sono definite nel file database.sql, presente nel repository.

4. Riepilogo e Repository GitHub

Voce	Dettaglio
Linguaggio	C++
Database	SQLite (tramite sqlite3.h)
Interfaccia	Console (testuale)
Codice sorgente	src/ (main.cpp + moduli separati)
File SQL	sql/database.sql + sql/popola_con_dati.sql
Demo video	demo/Demo.mp4
Documentazione	README.md

Repository GitHub del progetto:
https://github.com/csalvatore2/PersonalExpenseSystem